

Comprehensive Python Core Foundations: Complete Study Notes

This document provides complete, high-fidelity technical documentation covering the essential foundational pillars of Python. These core topics form the absolute baseline requirements for moving into advanced data pipeline engineering, machine learning workflows, and automated system development.

1. Python Basics & Control Flow

Python is a high-level, dynamically-typed language. Variables do not require explicit declaration of data types; instead, types are inferred dynamically at runtime.

Core Data Types

- **Numeric:** int (Integers), float (Floating-point decimal representations), complex (Complex numbers).
- **Sequence:** str (Immutable string sequences), list (Mutable ordered arrays), tuple (Immutable ordered structures).
- **Mapping & Set:** dict (Key-value pairs using hash tables), set (Unordered collection of unique items).

Conditional Logic & Loop Metrics

Control flow evaluates boolean logical operations sequentially utilizing if, elif, and else blocks. Iterations are achieved via:

- **For Loops:** Typically used to iterate over a bounded iterable collection or sequence range.
- **While Loops:** Continues execution indefinitely until a specific condition evaluates to False.

```
# Conditional and Loop paradigm
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        print(f"Even: {num}")
    else:
```

```
print(f"Odd: {num}")
```

2. Advanced Functions & Execution Scope

Functions are modular code structures defined using the `def` keyword. Python treats functions as first-class citizens, meaning they can be passed as arguments, returned from other functions, and assigned to variables.

Arguments and Parameter Types

- **Positional & Keyword Arguments:** Standard explicit naming parameters passed during a function call.
- **Arbitrary Arguments (*args):** Captures any positional arguments into an immutable tuple object.
- **Arbitrary Keyword Arguments (**kwargs):** Captures any keyword parameters into a standard mutable dictionary object.

Lambda Expressions

Lambda expressions are anonymous, single-line functions designed for inline usage inside operations such as mapping, filtering, or sorting.

```
# Variable length arguments combined with an inline lambda function
def flexible_logger(*args, **kwargs):
    print("Positional payload:", args)
    print("Keyword payload:", kwargs)
```

```
square_func = lambda x: x ** 2
print(square_func(5)) # Outputs 25
```

3. Object-Oriented Programming (OOP)

Object-Oriented Programming organizes code structures around self-contained entities called Classes (the blueprint) and Objects (instances of the blueprint).

The Four Pillars of OOP

1. **Inheritance:** Allows a child class to acquire the attributes and methods of a parent class, promoting code reuse.
2. **Polymorphism:** Allows different classes to define methods with identical names but unique functional behaviors (e.g., method overriding).
3. **Encapsulation:** Restricts direct external access to an object's internal variables. Indicated via a single underscore `_` (protected) or a double underscore `__` (private triggers name mangling).
4. **Abstraction:** Hides underlying complex implementation details, exposing only essential interface metrics via the `abc` module.

```
class ModelConfig:
    def __init__(self, learning_rate, batch_size):
        self._lr = learning_rate      # Protected parameter
        self.__bs = batch_size        # Private parameter

    def get_batch_size(self):          # Encapsulated getter
interface
    return self.__bs
```

4. File Handling & Stream Management

File operations require opening a stream connection to local storage, handling the operational read/write transaction, and safely closing the stream to prevent memory exhaustion or disk lock issues.

Standard Modes

- 'r': Read mode (default; errors if the target file does not exist).
- 'w': Write mode (truncates existing file contents or creates a new file).
- 'a': Append mode (adds data safely to the end of the file without overwriting).

Context Managers (The with Statement)

Using a context manager is the recommended best practice for file operations. It automatically manages resource teardown and guarantees that files are closed safely, even if an unhandled exception halts execution mid-stream.

```
# Safe stream context processing
with open('data_log.txt', 'w') as file_stream:
    file_stream.write("Initializing model deployment cycle...\n")
```

5. Robust Exception Handling

Runtime errors in Python trigger exceptions. If unhandled, these exceptions instantly abort application execution. Exception handling isolates blocks of vulnerable code to maintain system stability.

The Try-Except-Else-Finally Pipeline

- **try:** Houses the code block containing potential operational runtime failures.
- **except:** Catches and resolves specific targeted error exceptions.
- **else:** Runs only if no exceptions occur within the associated try block.
- **finally:** Executes unconditionally regardless of whether an exception occurred, making it ideal for resource cleanup.

```
try:
    result = 10 / 0
except ZeroDivisionError as err:
    print(f"Error caught gracefully: {err}")
finally:
    print("Execution pipeline cleanup completed.")
```

6. Modules & Packages Architectural Design

As code bases expand, keeping code organized requires splitting logic into distinct, manageable files and directories.

Definitions

- **Module:** Any standard Python file containing a .py extension, enclosing functions, variables, or classes meant to be shared across scripts.
- **Package:** A directory structure organizing multiple sub-modules. Packages typically

include an `__init__.py` file (which can remain empty) to signal to the interpreter that the directory should be treated as an importable module landscape.

```
# Module importation syntax variations
import math
from sklearn.metrics import accuracy_score
import numpy as np
```